1. Introdução

Operações de armazenagem em pilhas estão presentes em diversos ambientes logísticos de alta densidade, como pátios de terminais portuários, depósitos intermodais e sistemas automatizados de armazenagem vertical. Nesses contextos, contêineres ou blocos são organizados em pilhas verticais, isto é, uns sobre os outros. No entanto, somente o elemento localizado no topo de cada pilha pode ser removido diretamente. Quando o item solicitado não se encontra nessa posição, é necessário realocar os blocos que o bloqueiam antes de sua retirada.

Esse conjunto de movimentos constitui o foco do Problema de Realocação de Blocos (PRB, do inglês, *Block Relocation Problem*). Em sua variante restrita de descarregamento (PRB-D), somente os blocos posicionados diretamente acima do bloco alvo podem ser movimentados. O problema é classificado como NP-difícil em diversas de suas variantes (Lerstau e Shen, 2022), e o espaço de soluções cresce exponencialmente com o número de blocos e pilhas, inviabilizando a aplicação de métodos exatos para instâncias de porte moderado e grande.

A literatura sobre o PRB-D abrange uma ampla gama de abordagens. Métodos exatos, como o algoritmo *branch-and-bound* proposto por Kim e Hong (2006), garantem a otimalidade para instâncias de pequeno porte, mas apresentam limitações de escalabilidade. Heurísticas baseadas em busca em árvore ampliaram a capacidade de resolução, com destaque para os trabalhos de Forster e Bortfeldt (2012) e Zhu et al. (2012). Mais recentemente, métodos baseados em regras de vizinhança, como o VRH (do inglês, *Variable Reinsert Heuristic*) proposto por Ting e Wu (2017) e suas variantes consolidaram-se como referências de solução na literatura. Apesar desses avanços, a maior parte das abordagens existentes codifica a solução diretamente como uma sequência de movimentos ou como uma regra fixa de realocação, o que restringe a flexibilidade na exploração do espaço de soluções e dificulta a combinação de diferentes estratégias heurísticas em uma mesma abordagem.

Nesse cenário, representações de soluções por meio de chaves aleatórias apresentam potencial ainda pouco explorado para o PRB-D. Nessa classe de métodos, cada solução é codificada por um vetor de valores contínuos em *[0,1]*, cuja interpretação é delegada a um mecanismo denominado *decoder*. Essa separação entre busca e construção da solução permite que diferentes metaheurísticas operem sobre um espaço padronizado e contínuo, enquanto a lógica operacional do problema permanece encapsulada no *decoder*.

Motivado por esse cenário, este trabalho propõe uma abordagem baseada no *Random-Key Optimizer* (RKO), *framework* de otimização combinatória de código aberto proposto por

Chaves et al. (2025), para resolver o PRB-D. A contribuição reside na concepção de um *decoder* específico que transforma vetores de chaves aleatórias em ações de realocação factíveis, combinando três regras heurísticas distintas (orientadas, respectivamente, à preservação estrutural do pátio, ao balanceamento de ocupação das pilhas e à concentração controlada de blocos) cujas ativações são parametrizadas pelas chaves ao longo da otimização. Embora os experimentos computacionais indiquem que a abordagem proposta ainda não supera as metaheurísticas de referência em termos de qualidade de solução, o trabalho contribui para a prática da Pesquisa Operacional ao disseminar o *framework* RKO junto à comunidade de Engenharia de Produção, onde a ferramenta permanece pouco difundida, a despeito de seu potencial de aplicação a distintos problemas de otimização combinatória.

O restante do texto está organizado da seguinte forma. A Seção 2 apresenta uma revisão da literatura sobre o PRB-D e os trabalhos relacionados. A Seção 3 descreve a metodologia adotada, incluindo o *framework* RKO e o *decoder* proposto. A Seção 4 reporta os experimentos computacionais e discute o desempenho da abordagem frente aos métodos da literatura. Por fim, a Seção 5 apresenta as conclusões e aponta direções para pesquisas futuras.

2. Referencial teórico

O PRB tem sido amplamente estudado na literatura de Pesquisa Operacional desde o trabalho seminal de Kim e Hong (2006), motivado principalmente por aplicações em terminais portuários e sistemas de armazenagem de alta densidade. A classificação proposta por Lersteau e Shen (2022) organiza os métodos existentes em quatro grandes categorias: formulações matemáticas, heurísticas, metaheurísticas e métodos de busca em árvore. Embora as variantes restrita e irrestrita do problema sejam ambas NP-difíceis (Caserta, Schwarze e Voß, 2012), a literatura é significativamente mais desenvolvida para a variante restrita, que corresponde ao PRB-D abordado neste trabalho. Define-se a variante restrita pela condição de que apenas os blocos que bloqueiam o bloco-alvo atual podem sofrer realocação (ou seja, apenas movimentos forçados). Em contrapartida, a variante irrestrita prescinde dessa restrição, viabilizando movimentos antecipatórios

2.1 Métodos exatos e heurísticos

Os primeiros métodos exatos para o PRB-D foram baseados em algoritmos *branch-and-bound* (B&B). Kim e Hong (2006) propuseram um dos primeiros B&B para o problema, demonstrando sua aplicabilidade a instâncias de pequeno porte. Caserta, Schwarze

e Voß (2012) avançaram na direção das formulações de Programação Linear Inteira (PLI), propondo os modelos BRP-I, para a variante irrestrita, e BRP-II, para a restrita. Subsequentemente, Expósito-Izquierdo, Melián-Batista e Moreno-Vega (2015) identificaram inconsistências no modelo BRP-II e propuseram a versão corrigida BRP-II*, enquanto Zehendner et al. (2015) introduziram o modelo aprimorado BRP-II-A, reduzindo entre 65% e 89% das variáveis por meio de uma etapa de pré-processamento. Mais recentemente, Galle, Barnhart e Jaillet (2018) propuseram o modelo CRP-I, cuja reformulação com variáveis de adjacência entre itens resultou em menor número de variáveis e tempos computacionais superiores aos modelos anteriores. Tanaka e Voß (2022) apresentaram a abordagem exata de melhor desempenho conhecido para o PRB-D, reformulando o problema como a seleção de uma combinação ótima de sequências de realocação, resolvendo instâncias com até 100 itens de forma ótima dentro de uma hora.

No âmbito das heurísticas, Kim e Hong (2006) propuseram a regra ENAR (*Expected Number of Additional Relocations*), que estima o número de realocações adicionais esperadas para cada pilha candidata de destino. Caserta et al. (2012) introduziram a heurística MinMax, que tornou-se referência na literatura ao selecionar pilhas de destino priorizando aquelas que minimizam o impacto sobre blocos futuros; variantes e extensões dessa heurística foram propostas por Jovanovic e Voß (2014) e Galle et al. (2017). Métodos baseados em busca em árvore também se destacam: Forster e Bortfeldt (2012) desenvolveram um procedimento de busca em árvore com quatro componentes heurísticos integrados, enquanto Zhu et al. (2012) propuseram o algoritmo ID-A* (*Iterative Deepening A**), que combina limites inferiores progressivamente mais refinados com aprofundamento iterativo da busca. Ting e Wu (2017) consolidaram o estado da arte em heurísticas para o PRB-D ao propor a VRH (*Virtual Relocation Heuristic*) e suas variantes Complete-VRH, RF-VRH e AR-VRH, que determinam simultaneamente as pilhas de destino para todos os blocos que bloqueiam o bloco alvo atual, obtendo resultados de referência nas instâncias do *benchmark* ZQLZ (Zhu et al., 2012).

2.2 Metaheurísticas e representação de soluções

Entre as metaheurísticas aplicadas ao PRB-D, o Método do Corredor (*Corridor Method*) de Caserta e Voß (2009) restringe o espaço de busca a um subconjunto de pilhas candidatas e aplica busca local iterada sobre esse corredor, obtendo boas soluções em instâncias de maior porte. Jovanovic, Tuba e Voß (2019) propuseram um algoritmo de Otimização por Colônia de Formigas que associa trilhas de feromônio a tuplas compostas pelo bloco realocado, pela prioridade mínima da pilha de destino e pelo número de movimentos já realizados, superando

os métodos anteriores em qualidade de solução para instâncias de grande escala. ElWakil, Gheith e Eltawil (2019) aplicaram um *Simulated Annealing* ao PRB-D, obtendo resultados competitivos em instâncias pequenas, com desempenho inferior em instâncias maiores.

No que diz respeito à representação de soluções por chaves aleatórias, Hottung e Tierney (2016) aplicaram um BRKGA (*Biased Random-Key Genetic Algorithm*) ao Problema de Pré-Marshalling, que é um problema estruturalmente relacionado ao PRB-D, demonstrando o potencial dessa classe de representações em problemas de realocação de blocos.

Ao nosso melhor conhecimento, entretanto, o *framework* RKO, que generaliza e unifica diferentes metaheurísticas baseadas em chaves aleatórias em uma única infraestrutura de otimização, ainda não foi aplicado ao PRB-D.

3. Metodologia

A proposta deste trabalho consiste em desenvolver e avaliar uma abordagem baseada no *framework* RKO para o PRB-D, com foco na concepção de um *decoder* que transforme vetores de chaves aleatórias em operações de realocação factíveis.

A Pesquisa Operacional, ao empregar métodos quantitativos no suporte à tomada de decisões, fundamenta a base metodológica deste estudo, que se caracteriza como uma pesquisa quantitativa axiomática, estruturada a partir de uma simplificação teórica do problema real (Bertrand e Fransoo, 2002; Morabito e Pureza, 2010). A modelagem do problema e a concepção do *decoder* enquadram-se na pesquisa axiomática descritiva e normativa.

3.1. Descrição do problema

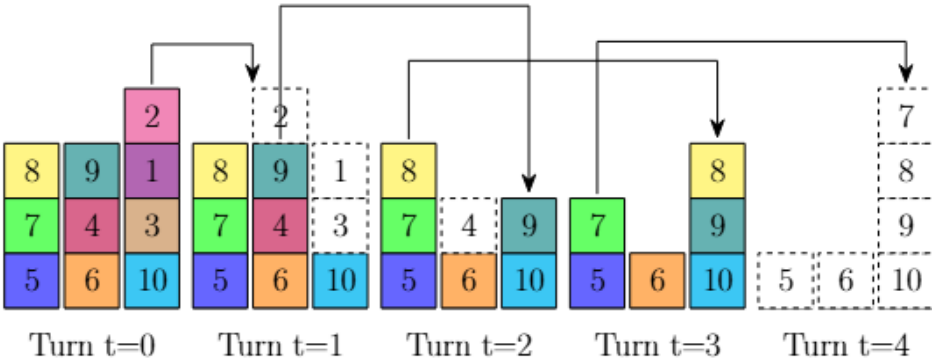
O PRB-D opera sobre um pátio composto por S pilhas, cada uma com capacidade máxima H (ou seja, cada pilha comporta verticalmente até H blocos). O pátio armazena um total de N blocos com prioridades distintas, identificadas por valores inteiros de 1 a N, em que 1 representa o bloco de maior prioridade. A retirada deve ocorrer em ordem crescente de prioridade. Denomina-se bloco-alvo aquele que deve ser removido do pátio no momento corrente. Quando o bloco-alvo não se encontra no topo de sua pilha, os blocos posicionados acima dele são considerados bloqueantes e devem ser realocados, individualmente, para o topo de outras pilhas com espaço disponível.

Conforme mencionado anteriormente, essa restrição caracteriza a variante restrita do problema, na qual são permitidas apenas realocações forçadas (movimentos de blocos que obstruem diretamente o bloco-alvo). O objetivo do PRB-D é minimizar o número total de realocações necessárias para esvaziar completamente o pátio. Embora parte dessas

realocações seja inevitável para viabilizar as retiradas, um planejamento inadequado pode gerar movimentos adicionais, sendo justamente esse excedente o foco da otimização.

A Figura 1 ilustra uma instância com $S = 3$ pilhas, $H = 4$ e N = 10 blocos. No estado inicial, a Pilha 1 contém os blocos 5 (base), 7 (meio) e 8 (topo); a Pilha 2 contém os blocos 6 (base), 4 (meio) e 9 (topo); e a Pilha 3 contém os blocos 10 (base), 3, 1 e 2 (topo). Para retirar o bloco 1 (de maior prioridade) é necessário realocar o bloco 2, que o bloqueia. Após relocar o bloco 2 da pilha 3 para a pilha 2, o bloco 1 passa a estar no topo da Pilha 3 e pode ser retirado imediatamente. Em seguida, o bloco 2 também pode ser removido por estar no topo de sua pilha, e assim por diante.

Figura 1 – Exemplo ilustrativo de uma instância do PRB-D com $S = 3$, $H = 3$ e $N = 10$.



Fonte: Elaborado pelos autores.

Como ilustrado na Figura 1, o estado do pátio é exibido antes de cada realocação. O termo *turn*, equivalente a "estágio" na literatura do PRB-D, indica o instante específico da captura, operando como um registro estático do depósito naquele momento.

3.2 *Framework* RKO

Chaves et al. (2025) estenderam a noção de chaves aleatórias, originalmente introduzidas no RKGA (do inglês, *Random-Key Genetic Algorithm*) e no BRKGA, para outras metaheurísticas ao propor o *framework* RKO.

O *framework* representa cada solução candidata por meio de um vetor de chaves aleatórias $x = (x_1, x_2, ..., x_n) \in [0,1]^n$, onde n é a dimensão do espaço de busca. A principal característica arquitetural do RKO é a separação explícita entre o mecanismo de busca e a construção da solução factível: enquanto o *framework* opera inteiramente no espaço contínuo $[0,1]^n$, a interpretação do vetor de chaves e a geração da solução correspondente são realizadas por um componente denominado *decoder*, específico ao problema e transparente ao processo de otimização.

No nível operacional, o RKO disponibiliza um conjunto de metaheurísticas que exploram e refinam os vetores de chaves ao longo da busca. Entre elas, destacam-se o BRKGA, que aplica cruzamento viesado entre soluções elite e não-elite; a Busca Local Iterada (ILS, do inglês, *Iterated Local Search*); e a Busca em Vizinhança Variável (VNS, do inglês, *Variable Neighborhood Search*).

A avaliação de cada vetor gerado ocorre em duas etapas: primeiro, o vetor é processado pelo *decoder*, que o converte em uma solução factível; em seguida, a qualidade dessa solução é medida pela função objetivo e o resultado é retornado ao mecanismo de busca para orientar as iterações subsequentes. Essa arquitetura modular torna o RKO especialmente adequado a problemas cujo processo decisório evolui dinamicamente ao longo da busca, pois permite encapsular toda a lógica operacional no *decoder* sem alterar a estrutura de busca.

3.3 *Decoder* proposto para o PRB-D

Para descrever o funcionamento do *decoder* proposto, define-se a seguinte notação:

- $p(b)$ denota a prioridade do bloco b, com $p(b) = 1$ para o primeiro bloco a ser retirado e $p(b) = N$ para o último;
- $pilha(b)$ indica a pilha que contém b no estado atual do pátio;
- $topo(s)$ designa o bloco no topo da pilha s;
- $|s|$ representa o número de blocos atualmente em s; *e,*
- $C = \{s \in \{1, ..., S\} : s \neq pilha(b) \wedge |s| < H\}$ é o conjunto de pilhas candidatas para receber um bloco realocado, isto é, excluem-se a pilha de origem e as pilhas já cheias.

Para representar uma solução, utiliza-se um vetor de N chaves aleatórias, em que N é o número de blocos. Dessa forma, cada posição do vetor está associada a uma regra de seleção de pilha de destino, definida conforme o valor contida nessa posição. Assim, o *decoder* consiste na simulação do processo de retirada em ordem de prioridade. Para cada bloco alvo b, percorrido de 1 até N, o algoritmo verifica se b está acessível no topo de sua pilha. Caso não esteja, um laço de desbloqueio é executado: o bloco no topo da pilha de b (denominado bloco bloqueador, denotado *bloq)* deve ser realocado para uma pilha de destino $dest \in C$.

A chave $x[bloq] \in [0,1]$ determina qual das três regras heurísticas é ativada para essa seleção. O movimento é aplicado, o contador de realocações R é incrementado, e o estado do pátio é atualizado. O laço se repete até que b esteja no topo de sua pilha e possa ser retirado. Ao final, o *decoder* retorna R como valor da função objetivo.

A Regra 1, ativada quando $x[bloq] < 0{,}33$, é orientada à preservação estrutural do pátio. Para cada pilha candidata $s \in C$, computa-se a pontuação de bloqueio $pontos(s) = |\{u \in s : p(u) <$

$p(bloq)\}|$, isto é, conta-se quantos blocos em cada s possuem prioridade estritamente maior que a de *bloq*. Em outras palavras, quantos blocos seriam retirados antes de *bloq* e, portanto, exigiriam sua remoção prévia se ele fosse depositado ali. A pilha de destino é aquela que minimiza essa pontuação, preferindo pilhas onde *bloq* não criará bloqueios futuros (ou em menor quantidade); empates são resolvidos pela menor ocupação.

A Regra 2, ativada quando $0{,}33 \leq x[bloq] < 0{,}66$, seleciona a pilha menos carregada de C (*argmin* $|s|$), promovendo distribuição homogênea das alturas e maior flexibilidade para realocações subsequentes; empates são resolvidos pelo menor índice de pilha.

A Regra 3, ativada quando $x[bloq] \geq 0{,}66$, adota a lógica oposta: seleciona a pilha mais carregada com espaço disponível (*argmax* $|s|$), concentrando blocos em poucas pilhas e preservando as demais como opções livres; empates são resolvidos pelo maior índice.

O Pseudocódigo 1 descreve a lógica completa do *decoder*, enquanto que as três funções auxiliares de seleção de pilha são descritas no Pseudocódigo 2.

Pseudocódigo 1 – *Decoder* proposto para o PRB-D.

	Algoritmo DECODER(x, instância)
	Entrada: *vetor de chaves x = (x₁, …, x□); configuração inicial do pátio*
	Saída: *R — número total de realocações realizadas*
1	`R ← 0`
2	`Inicializar estado do pátio conforme a instância`
3	`PARA b ← 1 ATÉ N FAÇA ▷ blocos em ordem crescente de prioridade`
4	`  ENQUANTO b ≠ topo(pilha(b)) FAÇA ▷ b está bloqueado`
5	`    bloq ← topo(pilha(b)) ▷ bloco bloqueador atual`
6	`    C ← {s ∈ {1,…,S} : s ≠ pilha(b) ∧ |s| < H} ▷ pilhas candidatas`
7	`    SE x[bloq] < 0,33 ENTÃO`
8	`      dest ← MÍNIMO_BLOQUEIOS(bloq, C)`
9	`    SENÃO SE x[bloq] < 0,66 ENTÃO`
10	`      dest ← MÍNIMA_OCUPAÇÃO(C)`
11	`    SENÃO`
12	`      dest ← MÁXIMA_OCUPAÇÃO(C)`
13	`    FIM SE`
14	`    Mover bloq para o topo de dest`
15	`    R ← R + 1`
16	`  FIM ENQUANTO`
17	`  Retirar b do topo de pilha(b) ▷ retirada sem custo adicional`
18	`FIM PARA`
19	`RETORNAR R`

Fonte: Elaborado pelos autores.

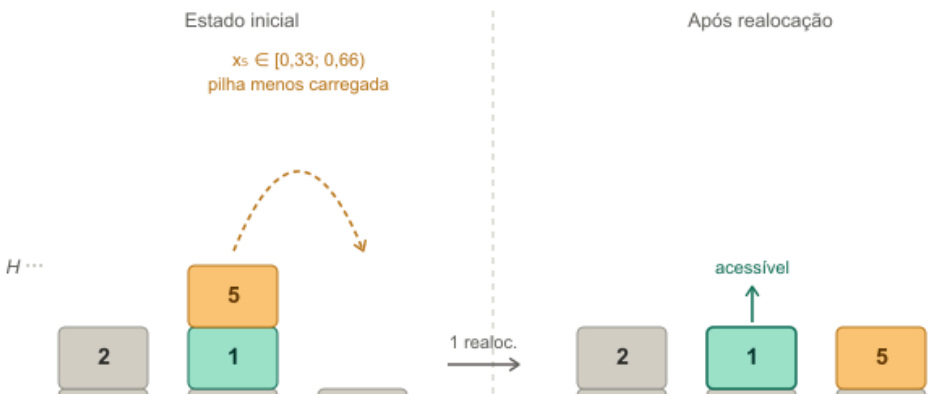
Pseudocódigo 2 – Regras de seleção de pilha de destino.

	Função MÍNIMO_BLOQUEIOS(bloq, C)
	Entrada: *bloco bloqueador bloq; conjunto de pilhas candidatas C*
	Saída: *pilha dest ∈ C com menor pontuação de bloqueio futuro*
20	**PARA CADA** s ∈ C **FAÇA**
21	pontos(s) ← |{u ∈ s : p(u) < p(bloq)}| ▷ *blocos com maior prioridade que bloq em s*
22	**FIM PARA**
23	**RETORNAR** argmin_{s∈C} pontos(s) ▷ *empate: menor |s|*
	Função MÍNIMA_OCUPAÇÃO(C)
	Entrada: *conjunto de pilhas candidatas C*
	Saída: *pilha dest ∈ C com menor número de blocos*
24	**RETORNAR** argmin_{s∈C} |s| ▷ *empate: menor índice de pilha*
	Função MÁXIMA_OCUPAÇÃO(C)
	Entrada: *conjunto de pilhas candidatas C*
	Saída: *pilha dest ∈ C com maior número de blocos*
25	**RETORNAR** argmax_{s∈C} |s| ▷ *empate: maior índice de pilha*

Fonte: Elaborado pelos autores.

Para ilustrar a execução do *decoder*, parte-se da instância da Figura 2 com $S = 3$, $H = 3$ e $N = 6$. Ao tentar retirar o bloco 1 ($b = 1$), o algoritmo encontra $bloq = 5$ bloqueando-o. O conjunto candidato é C = {Pilha 1, Pilha 3}. Aplicando a Regra 1: $pontos$(Pilha 1) = $|\{2, 4\} : p < 5| = 2$, enquanto $pontos$(Pilha 3) = $|\{6\} : p < 5| = 0$, isto é, $bloq$ seria enviado para a Pilha 3, sem criar bloqueios futuros. A Regra 2 também aponta para a Pilha 3, por ser a menos carregada ($|Pilha\ 3| = 1 < |Pilha\ 1| = 2$). Já a Regra 3 enviaria $bloq$ para a Pilha 1 (mais carregada), cobrindo os blocos 2 e 4, de maior prioridade, isto é, decisão localmente mais arriscada, mas que preserva a Pilha 3 desocupada para realocações futuras. Esse contraste evidencia como chaves distintas para o mesmo bloco bloqueador induzem comportamentos estruturalmente diferentes, e é justamente essa diversidade que o RKO explora ao variar os vetores ao longo da busca. Ressalta-se que a escolha da regra será determinada pelo valor da chave aleatória associada ao primeiro bloco.

Figura 2 – Exemplo ilustrativo de uma instância do PRB-D com S = 3, H = 3 e N = 6.



Fonte: Elaborado pelos autores.

4. Resultados e discussão

Esta seção reporta os experimentos computacionais realizados para avaliar o desempenho da abordagem proposta. A Seção 4.1 descreve a configuração experimental adotada, e a Seção 4.2 apresenta e discute os resultados obtidos.

4.1 Configuração experimental

Os experimentos foram conduzidos em um computador com processador Intel Core i5-9400F, 2,5GHz, 16GB de memória RAM e sistema operacional Windows 11. O *framework* RKO foi executado com um tempo limite de 10 segundos por instância e 100 execuções independentes. O decoder da Seção 3.3 foi implementado na versão Python para o RKO, disponível em <https://github.com/RKO-solver>.

As instâncias de *benchmark* utilizadas pertencem ao conjunto ZQLZ, proposto por Zhu et al. (2012) e adotado na literatura para avaliação de métodos para o PRB-D. Foram selecionados sete grupos de instâncias, identificados pela notação *H*-*S*-*N*, em que $H = 7$ denota a altura máxima das pilhas, $S = 10$ o número de pilhas e $N \in \{63, 64, 65, 66, 67, 68, 69\}$ o número de blocos. Cada grupo contém 100 instâncias geradas aleatoriamente com os mesmos parâmetros, isto é, tem-se um total de 700 instâncias. Os métodos de referência adotados para comparação são o algoritmo ID-A* de Zhu et al. (2012) e o algoritmo VRH de Ting e Wu (2017).

Para investigar a influência do mecanismo de busca sobre o desempenho do *decoder* proposto, avaliaram-se sete configurações do *framework* RKO: (i) apenas a metaheurística SA; (ii) apenas a metaheurística ILS; (iii) apenas a metaheurística PSO (do inglês, *Particle Swarm Optimization*); (iv) apenas a metaheurística BRKGA; (v) a combinação das metaheurísticas SA e ILS; (vi) a combinação das metaheurísticas PSO e BRKGA; e (vii) a combinação simultânea de todas as quatro metaheurísticas. Todas as configurações empregam o mesmo *decoder* descrito na Seção 3.3.

4.2 Experimentos computacionais

Para cada conjunto de instâncias, a Tabela 1 apresenta o número médio de realocações obtido pelo *decoder* proposto nas sete configurações do RKO sobre os grupos de instâncias ZQLZ selecionados.

Tabela 1 – Número médio de realocações por configuração do RKO com o *decoder* proposto.

Conjunto de instâncias	SA	ILS	PSO	BRKGA	SA+ILS	PSO + BRKGA	SA+ILS+ PSO+BR KGA
7-10-63	72,54	66,28	64,89	89,14	65,49	62,89	61,61
7-10-64	75,17	67,65	67,41	92,23	67,77	64,87	63,89
7-10-65	77,35	70,79	69,43	94,71	70,30	67,16	66,04
7-10-66	76,83	70,87	69,91	93,32	70,20	67,39	66,29
7-10-67	80,44	73,75	72,49	97,12	72,52	70,14	68,81
7-10-68	81,17	74,32	72,80	98,28	73,49	70,64	69,46
7-10-69	83,15	77,23	75,84	100,41	76,19	73,08	72,30
Média	**78,09**	**71,56**	**70,40**	**95,03**	**70,85**	**68,02**	**66,91**

Fonte: Elaborado pelos autores.

A Tabela 2 apresenta o número médio de realocações e o tempo médio em segundos obtido pelo *decoder* proposto nas sete configurações do RKO sobre todos os conjuntos de instâncias (7-10-63 até 7-10-69).

Tabela 2 – Número médio de realocações por configuração do RKO com o *decoder* proposto.

Metaheurísticas selecionadas do RKO	Número médio de relocações	Tempo médio (s)
SA	78,09	11,16
ILS	71,56	11,08
PSO	70,40	11,07
BRKGA	95,03	1,13
SA+ILS	70,85	13,18
PSO+BRKGA	68,02	21,15
SA+ILS+PSO+BRKGA	66,91	31,04

Fonte: Elaborado pelos autores.

A análise das Tabelas 1 e 2 indica que, dentre as configurações com metaheurística única, o PSO apresentou o menor número médio de realocações, atingindo 70,40 em média sobre os sete grupos, seguido de perto pelo ILS (71,56) e pelo SA (78,09). O BRKGA registrou o pior desempenho individual, com média de 95,03 realocações, isto é, um desempenho consideravelmente inferior ao das demais configurações isoladas, o que se reflete também em seu tempo médio de execução de apenas 1,13 s, sugerindo que a busca encerra prematuramente sem explorar adequadamente o espaço de chaves. No que diz respeito às configurações combinadas, a junção de SA+ILS resultou em média de 70,85 realocações (ligeiramente superior à do PSO isolado), indicando que a combinação dessas duas metaheurísticas não promove diversificação suficiente do espaço de busca para superar individualmente o PSO. Por outro lado, a combinação PSO+BRKGA obteve média de 68,02 realocações, superando todas as configurações individuais e sugerindo que a associação de metaheurísticas com características complementares potencializa a exploração do espaço de chaves. A configuração com todas as quatro metaheurísticas simultâneas

(SA+ILS+PSO+BRKGA) obteve a melhor média geral, com 66,91 realocações, representando uma redução de aproximadamente 5% em relação ao melhor resultado individual (PSO). De modo geral, o crescimento de *N* entre os grupos impacta de forma consistente os resultados de todas as configurações, com aumento médio de aproximadamente 16% nas realocações ao se passar do grupo 7-10-63 para o 7-10-69.

A Tabela 3 compara o melhor resultado obtido pela metaheurística proposta (configuração do *decoder* com SA+ILS+PSO+BRKGA) com os métodos de referência da literatura. Para a metaheurística proposta, são reportados o número total de realocações por grupo e o tempo médio de execução; para os métodos da literatura, os valores foram obtidos das publicações originais.

Tabela 3 – Comparação entre o melhor resultado da metaheurística proposta e os métodos de referência da literatura.

	Zhu et al. (2012)		**Ting e Wu (2017)**		**Metaheurística Proposta**	
Conjunto de instâncias	**Número total de relocações**	**Tempo médio (s)**	**Número total de relocações**	**Tempo médio (s)**	**Número total de relocações**	**Tempo médio (s)**
7-10-63	5912	0.016	5748	0.031	6161	31.10
7-10-64	6111	0.016	5963	0.036	6389	31.21
7-10-65	6386	0.016	6277	0.032	6604	31.07
7-10-66	6307	0.016	6206	0.063	6629	30.98
7-10-67	6668	0.015	6516	0.031	6881	30.98
7-10-68	6706	0.031	6313	0.047	6946	30.96
7-10-69	6964	0.020	6811	0.030	7230	30.96
Médias	6436.29	0.02	6262.00	0.04	6691.43	31.04

Fonte: Elaborado pelos autores.

A análise da Tabela 3 indica que o melhor resultado da metaheurística proposta apresenta uma média de 6.691 realocações totais, representando um *gap* de aproximadamente 7% em relação à abordagem de Ting e Wu (2017), de melhor desempenho entre os métodos de referência, e de aproximadamente 4% em relação ao ID-A* de Zhu et al. (2012). Esse resultado era esperado, dado que o *decoder* proposto opera sobre uma parametrização indireta do pátio, sem incorporar explicitamente os mecanismos de *lookahead* que caracterizam as heurísticas de referência.

Salienta-se, contudo, que a comparação com os resultados da literatura não é direta, uma vez que os experimentos de Zhu et al. (2012) e Ting e Wu (2017) foram conduzidos em configurações de hardware distintas das adotadas neste trabalho. O desempenho da metaheurística proposta em termos de tempo de execução, de 31,04 segundos em média para a melhor configuração, demonstra que a abordagem é computacionalmente viável dentro do

limite estabelecido, confirmando uma das vantagens práticas do *framework*: a capacidade de gerar soluções factíveis em tempo controlado para instâncias de porte moderado.

Como direção imediata de melhoria, o aprimoramento do decoder (por exemplo, mediante a incorporação de regras de *lookahead* ou a calibração adaptativa dos limiares de partição das chaves) representa o principal vetor para redução do gap observado, sem necessidade de modificar a estrutura de busca.

5. Conclusão

Este trabalho propôs e avaliou uma abordagem baseada no *framework* RKO para o Problema de Realocação de Blocos com descarregamento restrito (PRB-D), centrada na concepção de um *decoder* que combina três regras de seleção de pilha orientadas, respectivamente, à preservação estrutural do pátio, ao balanceamento de ocupação e à concentração controlada de blocos, cujas ativações são parametrizadas pelas chaves ao longo da otimização. Sete configurações do RKO foram avaliadas sobre instâncias do *benchmark* ZQLZ (Zhu et al., 2012).

Os experimentos indicam que a combinação simultânea de todas as quatro metaheurísticas (SA+ILS+PSO+BRKGA) apresentou o menor número médio de realocações entre as configurações avaliadas, atingindo 66,91 em média sobre os grupos de instâncias analisados. As configurações combinadas superaram as individuais de forma crescente conforme o número de metaheurísticas aumentou, sugerindo que a diversidade de mecanismos de busca favorece a exploração do espaço de chaves e, por consequência, a qualidade das soluções geradas pelo decoder. Em termos de eficiência computacional, todas as configurações operaram dentro do limite de tempo estabelecido, confirmando a viabilidade prática do framework para instâncias de porte moderado. Em termos de qualidade de solução, o melhor resultado do RKO apresenta um gap de aproximadamente 7% em relação à abordagem de Ting e Wu (2017), referência na literatura. Esse resultado era esperado: as heurísticas de referência incorporam mecanismos de bloqueio simultâneo e *lookahead* calibrados especificamente para o PRB-D, ao passo que o RKO delega integralmente as decisões de realocação ao decoder, sem acesso direto à estrutura do problema durante a busca.

Entende-se que o trabalho apresenta duas contribuições para a prática da Pesquisa Operacional. A primeira é a demonstração da aplicabilidade do *framework* RKO a um problema logístico real, contribuindo para sua disseminação junto à comunidade nacional de Engenharia de Produção, onde a ferramenta permanece pouco difundida a despeito de seu

potencial de generalização. A segunda é a introdução de um *decoder* interpretável: cada decisão de realocação pode ser explicitamente associada à regra heurística ativada, o que facilita a análise do comportamento do método e a proposição de extensões.

Como perspectivas de pesquisa futura, destacam-se o aprimoramento do decoder mediante regras adaptativas ou mecanismos de *lookahead* inspirados na família VRH, a calibração automática dos limiares de partição das chaves e a extensão da abordagem à variante irrestrita do PRB-D ou a problemas relacionados, como o Pré-Marshalling.

6. Agradecimentos

Agradecemos a Fundação de Amparo à Pesquisa do Estado de São Paulo (processos #2022/05803-3 e #2025/11059-3) e ao Conselho Nacional de Desenvolvimento Científico e Tecnológico (processos #408722/2023-1 e #402492/2025-0) pelo apoio financeiro.

REFERÊNCIAS

BERTRAND, J. W. M.; FRANSOO, J. C. Operations management research methodologies using quantitative modeling. **International Journal of Operations & Production Management**, v. 22, n. 2, p. 241–264, 2002.

CASERTA, M.; SCHWARZE, S.; VOß, S. A mathematical formulation and complexity considerations for the blocks relocation problem. **European Journal of Operational Research**, v. 219, n. 1, p. 96–104, 2012.

CASERTA, M.; VOß, S. Corridor selection and fine tuning for the corridor method. In: INTERNATIONAL SYMPOSIUM ON STOCHASTIC ALGORITHMS: FOUNDATIONS AND APPLICATIONS, 5., 2009, Sapporo. **Lecture Notes in Computer Science**, v. 5851. Berlin: Springer, 2009. p. 163–175.

CHAVES, A. A.; RESENDE, M. G. C.; SCHUETZ, M. J. A.; BRUBAKER, J.; KATZGRABER, H.; ARRUDA, E.; SILVA, R. A Random-Key optimizer for combinatorial optimization. **Journal of Heuristics**, v. 31, p. 32, 2025. DOI: 10.1007/s10732-025-09568-z.

ELWAKIL, M.; GHEITH, M.; ELTAWIL, A. A new simulated annealing based method for the container relocation problem. In: INTERNATIONAL CONFERENCE ON CONTROL, DECISION AND INFORMATION TECHNOLOGIES (CODIT), 6., 2019, Athens. **Proceedings...** [S.l.]: IEEE, 2019. p. 1432–1437.

EXPÓSITO-IZQUIERDO, C.; MELIÁN-BATISTA, B.; MORENO-VEGA, J. M. An exact approach for the blocks relocation problem. **Expert Systems with Applications**, v. 42, n. 17-18, p. 6408–6422, 2015.

FORSTER, F.; BORTFELDT, A. A tree search procedure for the container relocation problem. **Computers & Operations Research**, v. 39, n. 2, p. 299–309, 2012.

GALLE, V.; MANSHADI, V. H.; BORJIAN BOROUJENI, S.; BARNHART, C.; JAILLET, P. The stochastic container relocation problem. **Transportation Science**, v. 52, n. 5, p. 1035–1058, 2018. DOI: 10.1287/trsc.2018.0828.

GALLE, V.; BARNHART, C.; JAILLET, P. A new binary formulation of the restricted container relocation problem based on a binary encoding of configurations. **European Journal of Operational Research**, v. 267, n.

2, p. 467–477, 2018.

HOTTUNG, A.; TIERNEY, K. A biased random-key genetic algorithm for the container pre-marshalling problem. **Computers & Operations Research**, v. 75, p. 83–102, 2016.

JOVANOVIC, R.; TUBA, M.; VOß, S. An efficient ant colony optimization algorithm for the blocks relocation problem. **European Journal of Operational Research**, v. 274, n. 1, p. 78–90, 2019.

JOVANOVIC, R.; VOß, S. A chain heuristic for the blocks relocation problem. **Computers & Industrial Engineering**, v. 75, p. 79–86, 2014.

KIM, K. H.; HONG, G.-P. A heuristic rule for relocating blocks. **Computers & Operations Research**, v. 33, n. 4, p. 940–954, 2006.

LERSTEAU, C.; SHEN, W. A survey of optimization methods for block relocation and premarshalling problems. **Computers & Industrial Engineering**, v. 172, p. 108529, 2022.

MORABITO, R.; PUREZA, V. Modelagem e simulação. In: MIGUEL, P. A. C. (Org.). **Metodologia de pesquisa em engenharia de produção e gestão de operações**. Rio de Janeiro: Elsevier, 2010. cap. 8.

TANAKA, S.; VOß, S. An exact approach to the restricted block relocation problem based on a new integer programming formulation. **European Journal of Operational Research**, v. 296, n. 2, p. 485–503, 2022.

TING, C.-J.; WU, K.-C. Optimizing container relocation operations at container yards with beam search. **Transportation Research Part E: Logistics and Transportation Review**, v. 103, p. 17–31, 2017.

ZEHENDNER, E. et al. An improved mathematical formulation for the blocks relocation problem. **European Journal of Operational Research**, v. 245, n. 2, p. 415–422, 2015.

ZHU, W. et al. Iterative deepening A* algorithms for the container relocation problem. **IEEE Transactions on Automation Science and Engineering**, v. 9, n. 4, p. 710–722, 2012.